

Performance: Backend Spring Boot e serviços AWS

Anotações práticas para baixa latência e bom throughput — 2026-07-28

1. Visão geral

Performance boa entre um backend Spring Boot e a AWS depende menos de otimizar código isolado e mais de: mesma região, menos idas à rede, clientes reutilizados, batch, cache e trabalho assíncrono fora do request HTTP.

2. Latência de rede e topologia

- **Mesma região** do app e dos serviços (ex.: sa-east-1 se o público é BR).
- App em VPC privada: use **VPC Endpoints** (Gateway S3/DynamoDB; Interface para SSM, Secrets Manager, SQS) para não sair pela internet.
- Coloque ALB/API e ECS/EKS/EC2/Lambda na mesma região dos dados (RDS, DynamoDB, ElastiCache).
- Evite APIs chatty: muitas chamadas pequenas custam mais que uma chamada batch.

3. Spring Boot: cliente AWS e conexões

3.1 AWS SDK v2 (recomendado)

- Use o **AWS SDK for Java 2.x** com clientes **singleton** (bean Spring) — nunca criar client por request.
- HTTP client com connection pooling (Apache ou Netty async).
- Timeouts explícitos (connect, call, socket) e retry só para erros transitórios (429/5xx/throttling).
- Credenciais: IAM Role da instância/task/pod ou OIDC — evite access key longa no código.

3.2 Exemplo de bean (conceitual)

```
@Bean
DynamoDbClient dynamoDbClient() {
    return DynamoDbClient.builder()
        .region(Region.of(System.getenv().getOrDefault("AWS_REGION", "sa-east-1")))
        .build();
}
```

4. DynamoDB

- Preferir **GetItem / BatchGetItem / Query**; evitar **Scan** em caminho quente.
- Chave de partição bem distribuída (sem hot key única).
- Itens pequenos; payloads grandes → S3 + ponteiro no item DynamoDB.
- TTL para estados efêmeros (cooldown, cache operacional).
- Meça p95 e ThrottledRequests no CloudWatch.
- Capacidade: on-demand para carga irregular; provisioned + autoscaling se previsível.

5. S3, SQS, SNS, EventBridge

- **S3**: multipart para arquivos grandes; presigned URL para o cliente não passar pelo backend.
- **SQS**: tire trabalho pesado do request (imports, relatórios, e-mail). Worker Spring (@SqsListener) ou Lambda.
- Batch receive (até 10 mensagens) e long polling (WaitTimeSeconds=20).
- **SNS/EventBridge**: fan-out de eventos; não use request sync para notificar vários sistemas.

6. RDS / Aurora (SQL)

- Pool HikariCP dimensionado (não abrir conexão por request manualmente).
- Índices nas queries reais (EXPLAIN); evite N+1 no JPA (join fetch / EntityGraph).
- Read replica para leituras pesadas; writer só para escrita.
- Mesma AZ/região da app; SSL com truststore válido.

7. Cache

- Spring Cache + **ElastiCache (Redis)** para dados quentes (catálogo, tokens SSM com TTL).
- Cache local Caffeine para dados quase estáticos (segundos) — reduz ida ao Redis/Dynamo.
- Invalide de forma explícita após escrita; não confie só em TTL longo.

8. Arquitetura de request

- Controller fino: validação + orquestração; I/O AWS em services.
- Operações demoradas: HTTP 202 + id de job na fila SQS.
- Paralelize chamadas independentes com CompletableFuture / virtual threads (Java 21), com cuidado de saturação do pool.
- Circuit breaker (Resilience4j) para APIs externas e AWS em degradação.

9. Secrets e configuração

- Secrets Manager / SSM Parameter Store com **cache** no Spring (não GET a cada request).
- Spring Cloud AWS ou AWS SDK direto; refresh controlado.

10. Observabilidade

- Micrometer + CloudWatch (ou Datadog / X-Ray / OpenTelemetry).
- Trace o caminho HTTP → DynamoDB/S3/SQS; alerte p95 e throttling.
- Logs estruturados com correlation id (MDC).

11. Checklist rápido

- Região única e VPC endpoints quando aplicável
- Clientes AWS como beans singleton
- Timeouts + retry sensato
- Batch / Query em vez de Scan ou N+1
- Cache (Redis/Caffeine) no caminho quente
- Fila SQS para trabalho pesado
- Métricas p95 e throttling

Documento gerado para referência de projeto Spring Boot + AWS. Adapte pool, timeouts e capacidade ao perfil real de carga.